

Exploring and Improving Fault Localization in LLM Inference Engines

Tianhao Gu

02.2026–08.2026

Software Engineering & AI (SEAI)

Examiner: Prof. Dr. Chunyang Chen

Supervisor: Dr. Mark Huasong Meng

FROM FAILURE TO FAULT LOCATION

•  Issue report from GitHub

inference-engine /

└ server /

└ runner /

└ backend /

└ CMakeLists.txt

Outline

- 01** **Research Background**
Why is fault localization in LLM inference engines important and challenging?

- 02** **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL

- 03** **RQ2 — Why do LLM-agent-based methods fail in some cases?**
Are there recurring failure patterns?

- 04** **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow address observed failure patterns?

- 05** **Contribution and Discussion**

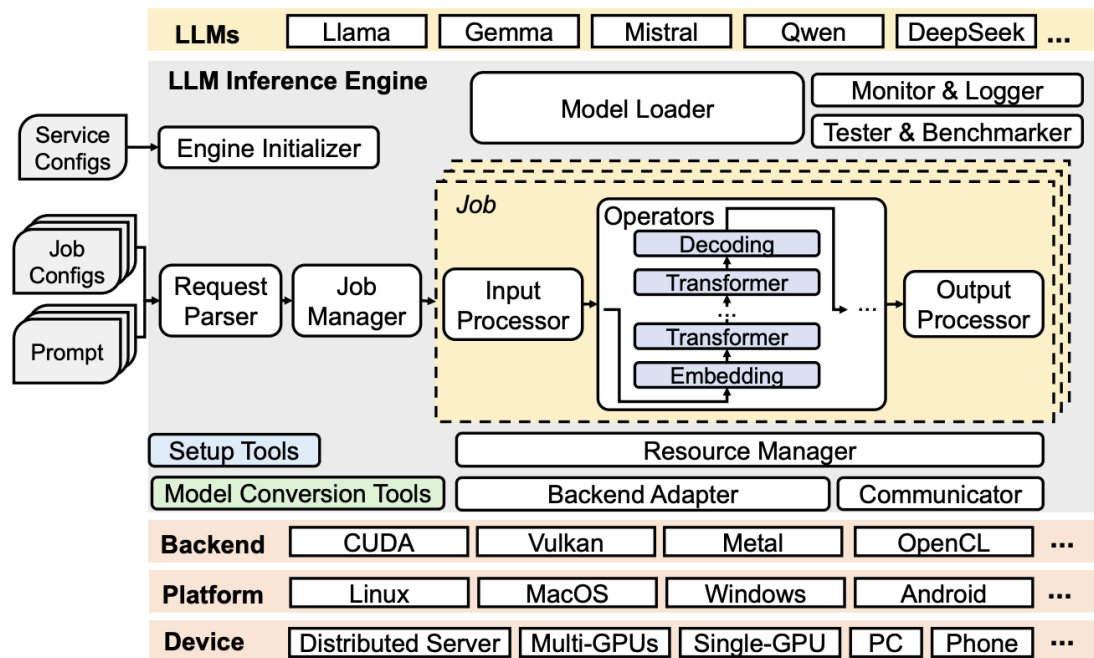
PART I

Research Background

Why is fault localization in LLM inference engines important and challenging?

How is an LLM deployed?

LLM inference engines have complex infrastructure.



Architecture overview [1]

The complex infrastructure coordinates request handling, model execution, resource management, backend adapters, and heterogeneous devices.

MODULE STRUCTURE

Interdependent modules across the serving stack

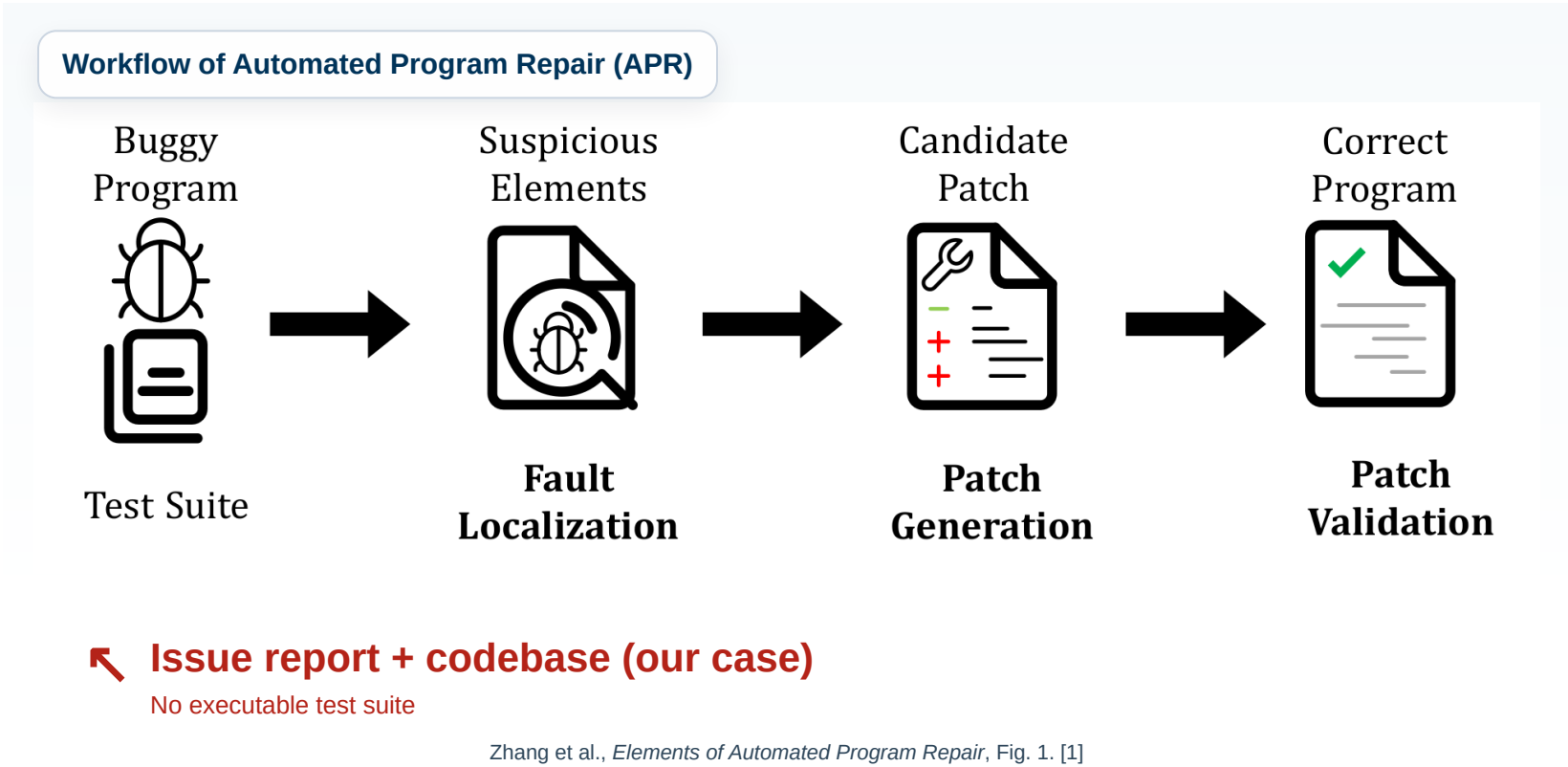
→ *in which module does the fault actually lie in?*

LIFECYCLE SCOPE

Spans setup, conversion, and LLM inference

→ *which phase is the starting point of the failure?*

What is Fault Localization?



FAULT LOCALIZATION

“Given the bug report and codebase, FL techniques identify buggy code elements (e.g., files or functions).”

Definition from the 2025 FL survey [1].

Importance

Failure of FL can directly cause failure of the whole APR process.

APR needs to localize the bug before patch generation and validation.

Why is FL hard for LLM inference engines?

Scale

1,254

tracked files · 324k LOC

The candidate space is too large for a single LLM context.

The repository cannot be fed directly into one prompt, so file-level localization must first narrow the search space before detailed reasoning begins.

Heterogeneous bug types

28

root-cause categories ^[1]

Symptoms and repair locations can live in different layers.

Failures span configuration, runtime, I/O, and functional logic, so the issue description often points to a visible symptom rather than the actual file that needs to be fixed.

Reproduction constraints

0

guaranteed executable oracle

Reliable reproduction evidence is often missing.

Many cases depend on specific hardware, drivers, models, or deployment environments, and issues often lack failing tests, complete traces, or stable reproduction steps.

Overview of existing fault-localization techniques.

LLM-based FL

e.g., LLMAO, AutoFL

Feeds the bug report and large spans of source code directly into one LLM call to score suspicious files or lines.

[5]

✗ Excluded in this setting — a full repository exceeds the usable context budget for single-pass prompting.

SBFL

/

MBFL

SBFL

Correlates pass/fail test-coverage spectra with suspicious statements.

MBFL

Compares mutant-kill patterns between passing and failing tests.

[6] [7]

✗ Excluded — requires an executable test suite

IRFL

Information-Retrieval Fault Localization

Ranks repository files by text and structural similarity between the bug report and the source code — no execution needed.

[1] [2]

✓ Feasible — repository text only

LLM-agent-based FL

e.g., SWE-agent, AutoCodeRover

An LLM autonomously searches, opens, and reasons over the repository, then proposes candidate files from the issue alone.

[3] [4]

✓ Feasible — no execution required

Research questions.

RQ1

Feasibility Exploration — Despite the aforementioned challenges, are representative IR-based and LLM-agent-based approaches feasible for execution-free file-level localization of the issues in the LLM inference engines?

Assessment: baseline feasibility

RQ2

Failure Explainability — Do LLM-agent localization failures in LLM inference engines reduce to a small set of recurring, identifiable pitfalls?

Analysis: recurring failure patterns

RQ3

Domain-Specific Improvement — Can incorporating domain-specific knowledge and a designed workflow address observed failure patterns?

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL
- 03 **RQ2 — Why do LLM-agent-based methods fail in some cases?**
Are there recurring failure patterns?
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow address observed failure patterns?
- 05 **Contribution and Discussion**

NEXT

PART II

RQ1 · Feasibility

Are the existing FL methods feasible?

Why Ollama as the studied inference engine?

ADOPTION

Widely used in practice

8.9M

developers served

Ollama is a live, widely adopted inference engine rather than a toy benchmark or research prototype.

FULL WORKFLOW COVERAGE

It covers the whole inference-engine workflow.

model acquisition

model management

API serving

request orchestration

resource-aware scheduling

inference runners

hardware backends

build and deployment artifacts

REAL-WORLD ENGINEERING

Its repository reflects production complexity.

Go, **C/C++**, and **TypeScript** coexist with configuration, build, and workflow files, which makes it a realistic study target for repository-scale fault localization.

WHY OLLAMA

A representative real-world case study for fault localization in LLM inference engines, not a synthetic benchmark.

1

Issue Collection

Mine closed, bug-labeled Ollama issues that were actually fixed by a merged pull request.

2

Fault Location Identification

Reconstruct the pre-fix snapshot and derive the exact repair-location label from the accepted fix.

3

Manual Inspection

Manually check every candidate so only clean, single-label instances remain.

Step 1 — Issue collection.

COLLECT

Start from Ollama's issue tracker

Pull closed issues that carry a bug label from the public Ollama GitHub repository.

CONFIRM THE FIX

Keep only issues closed by a merged PR

Retain issues closed via an explicit reference (fixes #N, closes #N, resolves #N) to a pull request, and confirm that PR was actually merged.

TWO EXCLUSIONS

— PR fixes more than one issue

— PR only creates new files

WHY EXCLUDE

A multi-issue PR cannot be mapped back to a single issue, and a newly created file has no pre-fix version to localize.

Step 2 — Fault-location identification.

GROUND TRUTH

The developer's own patch defines the label

The merged PR's patch is used as ground truth — it is what a real developer actually changed to fix the issue, not a guess or an approximation.



AVOID AMBIGUITY

Keep only a single ground-truth file **GT = 1**

If the patch's repair location resolves to more than one plausible file, the candidate is dropped — every retained instance has exactly one ground-truth repair location.

OUTPUT Each surviving candidate has exactly one reachable, patch-derived ground-truth repair-location label.

Step 3 — Manual inspection.

RELEVANCE

Is the PR actually about the issue?

Drop cases where the linked PR is unrelated or only mentions the issue in passing — a mislabeled link, not a real fix.

BUG VS. FEATURE

Is this really a bug?

Drop issues that are feature requests or enhancements mistakenly tagged as bugs.

INFORMATION LEAKAGE

Does the issue leak the fix?

Drop issues with too little diagnostic information to localize, or so much detail that they already reveal the repair location.

AI-ASSISTED RECOVERY

MISSED BUGS

Bugs never labeled "bug"

An LLM scans the repository for issues that describe real bugs but were never tagged with the bug label.

UNLINKED FIXES

Fixes GitHub never auto-linked

An LLM finds PRs that fix a bug issue but reference it in a format GitHub does not recognize, so no automatic issue–PR link is created.

RESULT

Manual and AI-assisted checks together keep OllamaFLBench clean and recover cases that label- and keyword-based mining alone would miss — 145 validated instances in total.

Studied Baselines.

Fault-location ranking results.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL
- 03 **RQ2 — Why do LLM-agent-based methods fail in some cases?**
Are there recurring failure patterns?
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow address observed failure patterns?
- 05 **Contribution and Discussion**

NEXT

PART III

RQ2 · Failure Mechanisms

Are there recurring failure patterns?

FAILURE PATTERN

Non-source under-inspection

Seeing a documentation or config reference is not the same as opening it — the repair file can sit in a linked artifact the agent never inspects.

4 of 145 cases

Top-5 ranking performance for non-source fault locations.

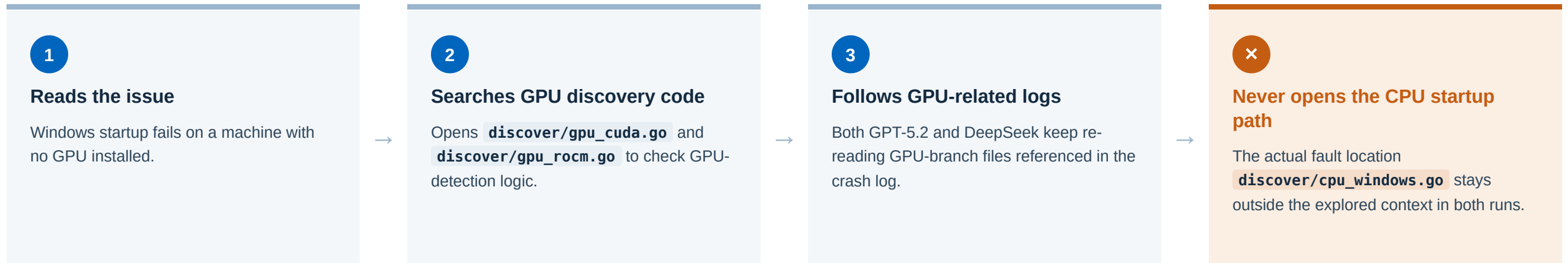
FAILURE PATTERN

Evidence non-reachability

A file that is never opened cannot be ranked — even when it sits right next to the code the agent does explore.

5 of 145 cases

Both agents get stuck in the GPU-discovery branch.



FAILURE PATTERN

Causal-chain truncation

When the fix crosses a language or process boundary, agents that stop exploring at that boundary miss it.

7 of 145 cases

GPT-5.2 stops one hop before the native producer.

1

Reads the issue

A streaming response is missing its final token.



2

Opens the Go-side scanning loop

Follows the token-stream parser inside the Go server package.



3

Confirms the Go logic looks complete

The scanning loop appears to read every chunk correctly.



✕

Stops at the language boundary

Never crosses into the native producer `llm/ext_server/server.cpp`, where the final token is actually dropped. DeepSeek V4 Pro continues past this boundary and ranks the file first.

Three recurring failure mechanisms.

CASE 126 · 5 OF 145 CASES

Evidence non-reachability

A neighboring path stays outside the explored context.

CASE 86 · 7 OF 145 CASES

Causal-chain truncation

The relevant chain crosses a language and process boundary.

CASE 103 · 4 OF 145 CASES

Non-source under-inspection

Seeing a reference is not the same as inspecting its evidence.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 · Feasibility**
IRFL vs LLM-agent-based FL
- 03 **RQ2 · Failure mechanisms**
Are there recurring failure patterns?
- 04 **RQ3 · Improvement**
Can domain knowledge and the designed workflow address observed failure patterns?
- 05 **Contributions & Discussion**

NEXT

PART IV

RQ3 · Improvement

How does ClueFL (our agent) improve baselines?

ClueFL uses repository evidence to rank candidate files.

Effect of domain guidance on fault-location ranking.

Effect of removing domain guidance.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1: Feasibility**
IRFL vs LLM-agent-based FL
- 03 **RQ2: Failure mechanisms**
Are there recurring failure patterns?
- 04 **RQ3: Improvement**
Can domain knowledge and the designed workflow address observed failure patterns?
- 05 **Contributions & Discussion**

NEXT



PART V

Contributions & Discussion

Contribution and Discussion

Conclusion and main findings.

IR

First systematic evaluation
Execution-free fault localization for LLM inference-engine bugs.

Agent

Repository-derived OllamaFLBench
145 validated Ollama issue–fix pairs with pre-fix snapshots.

ClueFL

Domain-specific ClueFL framework
External knowledge and workflow improve fault-location ranking.

Evaluate systematically → Release reusable data → Improve with domain knowledge